

Are You Smarter than DoRA?

Reproducing Weight-Decomposed Low-Rank Adaptation
on Competition Mathematics Benchmarks

Ryan Ye Boaz Ng Leon Jiao Aadi Singla
Cornell University, CS 4782 — Spring 2026

1 Introduction

Large language model fine-tuning is essential for specializing pre-trained models to new domains, but full fine-tuning updates billions of parameters at prohibitive cost. Parameter-Efficient Fine-Tuning (PEFT) methods address this by training only a small number of adapter parameters while keeping the base model frozen.

LoRA [2] is the most widely used PEFT approach: it adds a low-rank update $\Delta W = BA$ ($B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, $r \ll \min(d, k)$) to a frozen weight matrix W_0 . While effective, LoRA still shows a notable accuracy gap compared to full fine-tuning.

DoRA [1] closes this gap by decomposing W_0 into magnitude and direction, then applying LoRA only to the direction. The key insight from the paper is that full fine-tuning tends to make *large magnitude changes and small direction changes*, while LoRA can only make direction changes. By explicitly separating the two components, DoRA more closely mimics full fine-tuning behavior with the same parameter budget.

We re-implement DoRA from scratch in PyTorch and evaluate it on *competition mathematics*: AMC12 2024/2025 and the MATH benchmark — a new domain not studied in the original paper. We compare our implementation against PEFT-LoRA and PEFT-DoRA baselines, and introduce a three-mode evaluation protocol that reveals a novel “knows but can’t say” phenomenon in DoRA.

2 Chosen Result

We aim to reproduce the primary result of the DoRA paper: **DoRA consistently outperforms LoRA at the same rank/parameter budget** (paper Table 2, across NLP tasks including math and common-sense reasoning). A secondary claim is that DoRA at half rank ($r = 8$) matches full-rank LoRA ($r = 16$), demonstrating superior parameter efficiency.

We apply this to AMC12 2024/2025 (50 multiple-choice problems each, answers A–E) and the MATH benchmark (723 free-response problems across 5 dif-

ficulty levels). Our novel contribution is a three-scoring-mode evaluation protocol that decouples *knowledge* from *generation quality*, enabling finer-grained analysis than simple accuracy.

3 Methodology

3.1 DoRA Algorithm

DoRA adapts a frozen weight $W_0 \in \mathbb{R}^{d \times k}$ as:

$$W' = \mathbf{m} \cdot \frac{W_0 + s \cdot BA}{\|W_0 + s \cdot BA\|_c} \quad (1)$$

where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$ are low-rank LoRA factors initialized with $B = 0$ and A Kaiming uniform; $s = \alpha/r$ is a scaling factor; $\mathbf{m} \in \mathbb{R}^{1 \times k}$ is a trainable per-column magnitude vector initialized to $\|W_0\|_c$; and $\|\cdot\|_c$ denotes the column-wise ℓ_2 norm. Only $\mathbf{m}$, B , and A are trained; W_0 is frozen.

Critical implementation detail — the detach trick (Eq. 11 in [1]): the denominator $\|W_0 + s \cdot BA\|_c$ is *detached* before backpropagation. Without detaching, gradients flow through the norm, increasing memory by 24.4% on LLaMA-7B with only ~ 0.2 pp accuracy cost. Our implementation:

```
col_norm = weight_eff.norm(
    p=2, dim=0, keepdim=True
).detach().clamp_min(1e-12)
W_prime = (m / col_norm) * weight_eff
```

Merge/unmerge for zero inference cost: after training, we call `merge()`, which fuses W' back into the base layer weight and discards LoRA parameters, yielding a standard `nn.Linear` at inference with no latency overhead.

3.2 Implementation

We implement `DoRALinear`, a `nn.Module` that wraps a frozen `nn.Linear` layer and replaces its forward pass. The helper `apply_dora_to_module()` recursively traverses a model and replaces target layers by name. This approach is framework-agnostic and does not depend on PEFT’s internal structure.

We compare three methods:

- PEFT-LoRA:** `HuggingFace` `PEFT`
`LoraConfig(use_dora=False)`

2. **DoRA (PEFT):** HuggingFace PEFT LoraConfig(use_dora=True) — reference implementation
3. **Custom DoRA:** our from-scratch PyTorch implementation

Base model: Qwen2.5-3B (base and instruct variants). Target modules: `q_proj`, `k_proj`, `v_proj`, `up_proj`, `down_proj` — 5 of 7 attention/FFN projections. Total trainable parameters: $\sim 0.71\%$ of the model. Full implementation in Appendix A.

3.3 Datasets

Training tiers — all rows share the schema {problem, solution, answer, source}:

- **train_light:** MATH-lighteval [3] ($\sim 7,498$ rows; pure math: algebra, geometry, number theory, counting, precalc). Each row contains a full step-by-step solution ending in `\boxed{answer}`.
- **train_heavy:** `train_light` + NuminaMath-CoT [4] subset ($\sim 27,500$ total). NuminaMath adds physics, proof-style, and olympiad problems from a wider range of sources.
- **train_slk:** 1,000-row subset of `train_light`.

Example training rows:

MATH-lighteval (algebra):

Problem: In a 8 fl. oz. bottle of Vitamin Water, there are 125 calories. How many calories in a 12 fl. oz. bottle? **Answer:** 187.5

NuminaMath-CoT (geometry):

Problem: All vertices of right triangle ABC lie on $y=x^2$; hypotenuse AB is parallel to the x -axis. Prove the height $CD = 1$. **Answer:** 1

Leakage prevention: 4-stage pipeline (exact match $\rightarrow$ fuzzy 5-shingle Jaccard $\geq 0.6 \rightarrow$ answer cross-check $\rightarrow$ audit log); 0 problems dropped.

Evaluation datasets use different schemas:

AMC12 2024/2025 — schema {problem, answer, difficulty}, 50 problems per year (A–E multiple choice from official MAA exams):

Problem: What is the value of $9901 \cdot 101 - 99 \cdot 10101$? (A) 2 (B) 20 (C) 200 (D) 202 (E) 2020. **Answer:** A

MATH benchmark — schema {id, problem, solution, answer, level, type}, 723 problems across levels 1–5 and 7 types (algebra, geometry, number theory, etc.):

L1 (algebra): In a 8 fl. oz. bottle there are 125 calories; how many in 12 fl. oz.? **Ans:** 187.5

L3 (geometry): Kite $ABCD$ with $A(0,7)$, $B(1,0)$, $C(12,-2)$, $D(7,8)$; find area. **Ans:** 75

L5 (algebra): Evaluate $i^5 + i^{-25} + i^{45}$. **Ans:** i

3.4 Training Setup

We use TRL’s `SFTTrainer` for supervised fine-tuning on (problem, solution) pairs. Hyperparameters: rank $r = 8$, $\alpha = 32$, dropout = 0.05, learning rate 1×10^{-4} , cosine LR schedule, 3% warmup, effective batch size 16 (per-device 1, gradient accumulation 16), 1–3 epochs.

3.5 Novel Evaluation Protocol

Standard greedy accuracy conflates knowledge with generation quality. We introduce three scoring modes:

1. **Logit scoring:** append “\n\nThe answer is (” to the prompt $\rightarrow$ single forward pass $\rightarrow$ read logits for tokens A/B/C/D/E $\rightarrow$ argmax. A pure *knowledge probe*: does the model assign highest probability to the correct answer token?
2. **Two-pass scoring:** generate a full reasoning chain, then append the suffix and read answer logits. Tests whether reasoning improves answer selection.
3. **Greedy scoring:** generate a full response (up to 2048 tokens) and parse the `\boxed{}` answer. Standard *generation quality* metric.

This protocol lets us ask: does a model *know* the answer (logit) but *fail to say it* (greedy)? As we show below, this distinction is the central finding of our work. Figure 1 illustrates logit scoring visually. Implementation is in Appendix B.

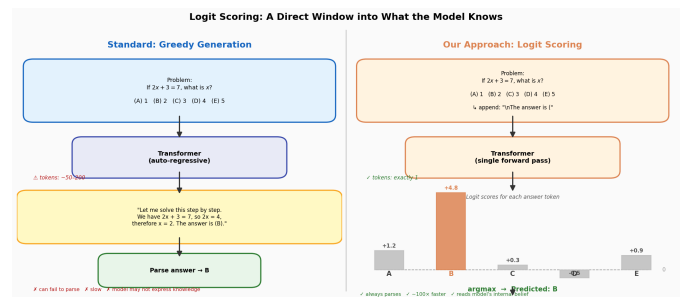


Figure 1: Logit scoring (right) reads the model’s internal answer belief with a single forward pass, vs. greedy generation (left) which requires parsing a full output sequence.

4 Results & Analysis

4.1 AMC12 2024 Main Results

Table 1 summarizes performance across all three scoring modes on AMC12 2024. Figure 2 shows the same results visually.

Table 1: AMC12 2024 accuracy by scoring mode. Δ = logit – greedy. Best per column in **bold**.

Model	Logit	2-pass	Greedy	Δ
Base (3B-base)	32%	—	—	—
Base (3B-inst)	34%	—	44%	−10pp
LoRA light (base)	38%	24%	18%	+20pp
LoRA light (inst)	44%	32%	10%	+34pp
LoRA heavy (base)	34%	32%	36%	−2pp
DoRA light (base)	38%	28%	12%	+26pp
DoRA light (inst)	36%	24%	8%	+28pp
DoRA heavy (base)	32%	—	46%	−14pp
DoRA r4 (base)	44%	—	16%	+28pp

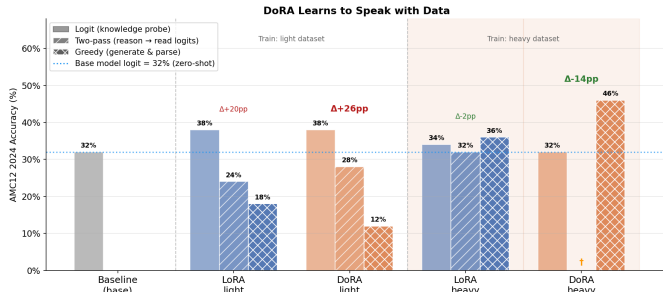


Figure 2: DoRA vs. LoRA on AMC12 2024 across all three scoring modes. DoRA’s logit–greedy gap (Δ) is consistently larger than LoRA’s. The base model logit (32%) is shown as a dotted reference line.

4.2 MATH Benchmark

Table 2 shows MATH benchmark accuracy. Figure 3 shows the breakdown by difficulty level.

Table 2: MATH benchmark accuracy (greedy). Best in **bold**.

Model	Overall	L1	L3	L5
Baseline (base)	43.8%	74.3%	57.6%	22.1%
Baseline (instruct)	52.7%	82.6%	66.1%	31.4%
LoRA s1k (inst)	44.8%	77.1%	56.5%	24.2%
LoRA heavy (base)	35.0%	73.4%	43.5%	15.1%
DoRA light (base)	30.7%	70.6%	36.4%	12.7%

4.3 Key Findings

Finding 1 — “DoRA knows but can’t say” (main finding). The logit–greedy gap is consistently and significantly larger for DoRA than LoRA

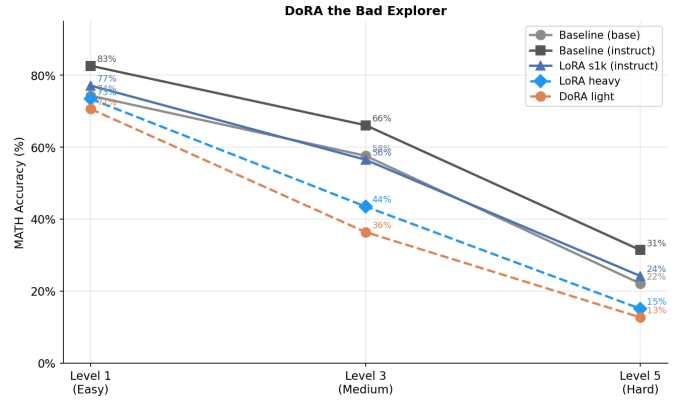


Figure 3: MATH accuracy by difficulty level (1=easy, 5=hard). DoRA light collapses fastest at hard problems. LoRA s1k (instruct, 1K training examples) tracks the baseline most closely.

at the same training configuration. DoRA light shows $\Delta = +26pp$ vs. LoRA light’s $\Delta = +20pp$. This means DoRA assigns high probability to the correct answer token internally, but fails to produce it through generation. We hypothesize that DoRA’s magnitude–direction decomposition reorganizes how the model stores knowledge in a way that makes it harder to verbalize, even when the representation is correct.

Finding 2 — More training data closes, and eventually reverses, the logit–greedy gap. Both LoRA and DoRA benefit from more data, but in different ways. LoRA heavy ($3.7\times$ more data) gradually closes its gap: logit $\approx$ greedy ($34\% \approx 36\%$, $\Delta = -2pp$). DoRA heavy shows a more dramatic shift: greedy (46%) *exceeds* logit (32%), $\Delta = -14pp$ — the gap fully reverses. DoRA heavy achieves the highest greedy accuracy of any fine-tuned model. Interestingly, while both methods move in the same direction (more data $\rightarrow$ greedy improves relative to logit), DoRA’s swing is much larger, going from $\Delta = +26pp$ (light) to $\Delta = -14pp$ (heavy). This suggests DoRA particularly benefits from additional training signal to learn fluent generation, while LoRA converges more smoothly.

Finding 3 — Knowledge is rank-invariant for DoRA. Across r2, r4, and r8, DoRA logit accuracy stays near 42–44%. Knowledge representation is captured regardless of rank. However, DoRA greedy accuracy remains low at all ranks (12–16%), while LoRA greedy improves more readily with rank. This suggests the logit–greedy gap is a property of DoRA’s adaptation mechanism, not its capacity.

Finding 4 — MATH regression is worse for

DoRA. Fine-tuning on AMC-style data hurts general math reasoning. DoRA light falls 13pp below baseline (30.7% vs. 43.8%), worse than LoRA heavy (35.0%). Notably, LoRA slk (1K-example instruct fine-tune) nearly preserves MATH (44.8%), suggesting small targeted datasets cause less catastrophic interference with general knowledge. DoRA’s magnitude decomposition may cause more aggressive weight modification, amplifying this effect.

Finding 5 — Partial replication of the paper. The original DoRA paper demonstrates DoRA > LoRA on 7B models across NLP tasks. At the logit (knowledge) level, we reproduce this: DoRA r4 matches the best LoRA logit result (both 44%). However, greedy accuracy does not confirm DoRA > LoRA at 3B scale. Likely causes: (a) model scale (3B vs. 7B in paper), (b) math is more domain-specific than commonsense, (c) our training data is smaller. The paper’s claims may require larger models to fully manifest.

5 Reflections

Logit scoring as a knowledge probe. The three-mode evaluation protocol was our most important methodological contribution. Standard greedy evaluation would have shown only that DoRA underperforms LoRA on generation. Logit scoring revealed that DoRA actually *learns the right answer* — it just cannot verbalize it. Two-pass scoring (generate reasoning, then read logits) provides an intermediate point: DoRA two-pass consistently falls between logit and greedy, suggesting reasoning chains partially help but do not fully close the generation gap. This insight opens a new research question: can logit-guided decoding recover DoRA’s latent knowledge?

Scale limitations. The original paper evaluates on 7B–13B models with large, high-quality datasets (e.g., MetaMathQA, commonsense datasets). At 3B with ~7.5K training problems, our setup is a harder regime where PEFT methods struggle more, and the gains from DoRA over LoRA may not yet be visible in generation. Our results suggest 3B is below the threshold where DoRA’s advantages over LoRA clearly manifest in generation quality.

Future directions.

1. **Larger models:** 7B+ to match the paper’s regime, where DoRA’s generation advantage may materialize.
2. **Harder benchmarks:** AIME 2024/2025 (30 free-response problems, much harder than AMC12).

3. **Decoding interventions:** given DoRA “knows but can’t say,” can logit-guided or constrained decoding recover the latent knowledge?
4. **Mechanistic understanding:** why does DoRA’s magnitude decomposition create a larger logit–greedy gap? Probing the magnitude vectors (**m**) may reveal which layers and features drive this behavior.

References

- [1] S.-Y. Liu, C.-Y. Wang, H. Yin, P. Molchanov, Y.-C. F. Wang, K.-T. Cheng, M.-H. Chen. *DoRA: Weight-Decomposed Low-Rank Adaptation*. 41st International Conference on Machine Learning (ICML), 2024.
- [2] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen. *LoRA: Low-Rank Adaptation of Large Language Models*. International Conference on Learning Representations (ICLR), 2022.
- [3] DigitalLearningGmbH. *MATH-lighteval dataset*. Hugging Face, 2024. <https://huggingface.co/datasets/DigitalLearningGmbH/MATH-lighteval>
- [4] J. Li et al. *NuminaMath-CoT dataset*. AI-MO / Hugging Face, 2024. <https://huggingface.co/datasets/AI-MO/NuminaMath-CoT>
- [5] S. Mangrulkar, S. Gugger, L. Debut, Y. Belkada, S. Paul, B. Bossan. *PEFT: State-of-the-art Parameter-Efficient Fine-Tuning methods*. Hugging Face, 2022. <https://github.com/huggingface/peft>
- [6] Qwen Team. *Qwen2.5: A Party of Foundation Language Models*. Alibaba Cloud, 2024. <https://huggingface.co/Qwen/Qwen2.5-3B>

A DoRA Implementation (code/dora_layers.py)

The full DoRALinear layer and model injection helper. The key equations are in `_compute_scaled_weight`: the LoRA update is added to the frozen base weight, the column-wise norm is computed and detached, and the magnitude vector `m` rescales the result.

```
1 class DoRALinear(nn.Module):
2     def __init__(self, base: nn.Linear, rank: int,
3                 alpha: float, dropout: float):
4         super().__init__()
5         self.base = base          # frozen pretrained weight W_0
6         self.scaling = alpha / rank # LoRA scaling factor s = alpha/r
7         in_f, out_f = base.in_features, base.out_features
8         dev, dtype = base.weight.device, base.weight.dtype
9
10        # Low-rank factors: B (d x r) init=zeros, A (r x k) Kaiming
11        # At init, BA=0 so W' = W_0 (no change until training starts)
12        self.lora_A = nn.Parameter(
13            torch.zeros((rank, in_f), device=dev, dtype=dtype))
14        self.lora_B = nn.Parameter(
15            torch.zeros((out_f, rank), device=dev, dtype=dtype))
16        nn.init.kaiming_uniform_(self.lora_A, a=0)
17
18        with torch.no_grad():
19            # m initialized to ||W_0||_c so W' = W_0 at step 0
20            self.m = nn.Parameter(
21                base.weight.norm(p=2, dim=0, keepdim=True).clone())
22
23        # Freeze base weight: only m, lora_A, lora_B are trainable
24        for p in self.base.parameters():
25            p.requires_grad = False
26        self.dropout = nn.Dropout(p=dropout)
27
28    def _compute_scaled_weight(self) -> torch.Tensor:
29        # Step 1: compute the effective weight W_0 + s*BA (Eq. 5 numerator)
30        lora_update = (self.lora_B @ self.lora_A) * self.scaling
31        weight_eff = self.base.weight + lora_update
32
33        # Step 2: column-wise L2 norm, detached (Eq. 11 detach trick).
34        # Detaching stops gradients flowing through the norm denominator,
35        # saving ~24% memory at ~0.2pp accuracy cost (per paper ablation).
36        col_norm = weight_eff.norm(
37            p=2, dim=0, keepdim=True
38        ).detach().clamp_min(1e-12) # clamp avoids division by zero
39
```

```

40     # Step 3: rescale by learned magnitude m --- Eq. 5
41     return (self.m / col_norm) * weight_eff
42
43 def forward(self, x: torch.Tensor) -> torch.Tensor:
44     if self.merged:
45         # After merge(), W' is already baked into base.weight
46         return self.base(x)
47     # Apply dropout to input, then matmul with DoRA-scaled weight
48     return F.linear(self.dropout(x),
49                     self._compute_scaled_weight(),
50                     self.base.bias)
51
52 def merge(self) -> None:
53     """Bake W' into base.weight so inference is a plain Linear."""
54     self._cached_base_weight = self.base.weight.detach().clone()
55     with torch.no_grad():
56         self.base.weight.copy_(self._compute_scaled_weight())
57     self.merged = True

```

```

1 def apply_dora_to_module(model, target_names,
2                           rank, alpha, dropout=0.05):
3     """Walk the model tree; swap matching Linear -> DoRALinear."""
4     target_set = set(target_names) # e.g. {"q_proj", "v_proj", ...}
5     def recurse(module, prefix=""):
6         for name, child in module.named_children():
7             qname = f"{prefix}.{name}" if prefix else name
8             if (isinstance(child, nn.Linear)
9                 and (name in target_set
10                     or qname in target_set)):
11                 # In-place replacement: parent.child = DoRALinear(child)
12                 setattr(module, name,
13                         DoRALinear(child, rank, alpha, dropout))
14             else:
15                 recurse(child, qname) # recurse into sub-modules
16     recurse(model)
17     return model

```

B Evaluation Scoring Modes (code/eval.py)

The three scoring functions used to decouple knowledge from generation quality.

```

1 MC_LETTERS = ["A", "B", "C", "D", "E"]
2 # Appending this suffix steers the model to predict a letter next
3 SUFFIX = "\nThe answer is ("
4
5 def _letter_logits(model, tokenizer, text):
6     """Core of logit scoring: one forward pass, read next-token logits."""
7     # Tokenize prompt + suffix; model sees the full context
8     inputs = tokenizer(text + SUFFIX,
9                        return_tensors="pt").to(model.device)
10    # Map each letter to its token ID (single-token for A-E)
11    letter_ids = {l: tokenizer.encode(
12        l, add_special_tokens=False)[0] for l in MC_LETTERS}
13    with torch.no_grad():
14        # logits[0, -1, :] = distribution over vocab at the *last* position
15        last_logits = model(**inputs).logits[0, -1, :]
16    # Extract raw logit (not probability) for each answer letter
17    scores = {l: last_logits[tid].item()
18              for l, tid in letter_ids.items()}
19    # argmax over A-E gives the model's most likely next token
20    return max(scores, key=scores.__getitem__), scores
21

```



```

22 def score_mc_logit(model, tokenizer, prompt):
23     """Logit scoring: no generation, pure knowledge probe."""
24     return _letter_logits(model, tokenizer, prompt)
25
26 def score_mc_twopass(model, tokenizer, prompt,
27                     max_new_tokens=4096):
28     """Two-pass: let model reason first, then read answer logits."""
29     inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
30     prompt_len = inputs["input_ids"].shape[1]
31     with torch.no_grad():
32         # Pass 1: generate a full reasoning chain (greedy)
33         out = model.generate(**inputs, do_sample=False,
34                             max_new_tokens=max_new_tokens)
35     # Decode only the newly generated tokens (skip the prompt)
36     reasoning = tokenizer.decode(out[0][prompt_len:],
37                                 skip_special_tokens=True)
38     # Pass 2: append reasoning to prompt, re-read answer logits
39     return _letter_logits(model, tokenizer, prompt + reasoning)
40
41 def generate_responses(model, tokenizer, prompt, n,
42                      temperature, max_new_tokens=4096):
43     """Greedy scoring: generate full response, caller parses \\boxed{\\}. """
44     inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
45     prompt_len = inputs["input_ids"].shape[1]
46     # Greedy (temp=0) or sampled (temp>0, for majority-vote pass@k)
47     gen_kwargs = (dict(do_sample=False, num_return_sequences=1)
48                  if n == 1 or temperature == 0.0
49                  else dict(do_sample=True, temperature=temperature,
50                            num_return_sequences=n))
51     with torch.no_grad():
52         out = model.generate(**inputs, **gen_kwargs,
53                             max_new_tokens=max_new_tokens)
54     # Slice off the prompt tokens; return only generated text
55     return [tokenizer.decode(seq[prompt_len:],
56                               skip_special_tokens=True)
57             for seq in out]

```